

C++ Template 自主學習報告

主題: STL 與基礎資料結構入門

小專題: 學生成績管理系統

姓名: 林承諺

科系: 資工一

課程: 計算機程式設計(二)

GitHub Repository:

<https://github.com/sodaoao/student-grade-management-system>

一、自主學習主題與目標

本次自主學習的主題是「STL 與基礎資料結構入門」，主要是延伸課堂 Chapter 16 Templates 的內容。課堂中 template 的概念是讓函式或類別可以套用在不同型別上，而 C++ 標準模板函式庫 STL 正是 template 的實際應用。

這次小專題的目標是設計一個「學生成績管理系統」，透過 C++ 程式管理學生的學號、姓名與成績。程式需要完成新增學生、列出學生、依成績排序、依學號查詢，以及統計平均分數、最高分、最低分、及格人數與不及格人數等功能。

透過這次作業，我希望能學會如何使用 STL 容器儲存資料，並理解 `vector`、`sort()`、`struct` 和 template function 如何整合在一個完整的小型程式中。

二、Template 與 STL 的關係

Template 是 C++ 中很重要的語法，可以讓函式或類別不只限制在單一資料型別。例如一般函式如果只能比較 `int`，那遇到 `double` 或其他型別時就需要重新寫一個函式。但使用 `template` 後，就可以讓同一個函式支援不同型別。

例如本次作業使用的 `getMax` 和 `getMin` 就是 `template function`：

```
8 // Template Function：取得較大值
9 template <class T>
10 T getMax(T a, T b) {
11     return (a > b) ? a : b;
12 }
13
14 // Template Function：取得較小值
15 template <class T>
16 T getMin(T a, T b) {
17     return (a < b) ? a : b;
18 }
```

這兩個函式可以用來比較兩個相同型別的資料。在本專題中，我將它們用於成績統計，協助計算最高分與最低分。

STL 是 Standard Template Library，也就是 C++ 標準模板函式庫。STL 裡面的許多容器，例如 `vector`、`stack`、`queue`、`map`、`set`，其實都是 `template` 的應用。因此 STL 可以儲存不同型別的資料，例如 `vector<int>` 可以存整數，`vector<string>` 可以存字串，而本專題使用的 `vector<Student>` 則可以存放學生資料。

三、常見 STL 容器用途整理

STL 提供了許多常見容器，每一種容器適合不同的使用情境。

`vector` 是動態陣列，可以依照需要新增資料，適合用來儲存一串資料。本次專題使用 `vector<Student>` 儲存多位學生資料，因為學生人數可能增加，使用 `vector` 比傳統陣列方便。

`stack` 是堆疊資料結構，特色是後進先出，也就是 LIFO。常見用途是括號配對、函式呼叫、回溯問題等。

`queue` 是佇列資料結構，特色是先進先出，也就是 FIFO。常見用途是排隊系統、服務櫃台、廣度優先搜尋等。

`map` 是鍵值對資料結構，可以用 key 對應 value。例如可以用學號當 key，學生資料當 value，快速查詢某個學生。

`set` 是集合資料結構，通常用來儲存不重複的資料。

`priority_queue` 是優先佇列，會根據優先順序取出資料，常用於需要快速取得最大值或最小值的情況。

這次專題我選擇使用 `vector`，因為本系統的功能需要新增學生、列出全部學生、排序學生資料，這些操作使用 `vector` 比較直覺，也比較適合目前的學習程度。

四、第 2 週小專題設計說明

本次小專題是「學生成績管理系統」。每位學生包含三個欄位：學號、姓名、成績，因此我使用 `struct Student` 來設計學生資料結構。

```
20 // 學生資料結構
21 struct Student {
22     string id;
23     string name;
24     int score;
25 };
```

程式主要使用選單介面，讓使用者可以輸入功能編號來操作系統。選單包含新增學生、列出所有學生、依成績排序、查詢學生、顯示統計資料與離開程式。

系統功能如下：

1. 新增學生資料：輸入學號、姓名與成績，並存入 `vector`。
2. 列出所有學生：顯示目前已輸入的所有學生資料。
3. 依成績排序：使用 `sort()` 將學生依成績由高到低排列。
4. 查詢學生：輸入學號後，尋找對應學生資料。
5. 統計成績：計算平均分數、最高分、最低分、及格人數與不及格人數。
6. 離開程式：結束系統。

本系統雖然功能簡單，但已經包含 STL 容器、STL algorithm、struct、function 和 template function，因此可以作為 C++ 資料結構入門的練習。

五、主要程式架構與重要程式片段

本專題的主要資料儲存方式是：

```
vector<Student> students;
```

這行程式代表建立一個可以存放多位學生的容器。每次新增學生時，就會使用 `push_back()` 將新的學生資料放進 `vector` 中。

新增學生的功能如下：

```
27 // 新增學生
28 void addStudent(vector<Student>& students) {
29     Student s;
30
31     cout << "Enter student id: ";
32     cin >> s.id;
33
34     for (const auto& student : students) {
35         if (student.id == s.id) {
36             cout << "This id already exists. Add failed.\n";
37             return;
38         }
39     }
40
41     cout << "Enter student name: ";
42     cin >> s.name;
43
44     cout << "Enter score: ";
45     cin >> s.score;
46
47     students.push_back(s);
48     cout << "Student added successfully.\n";
49 }
50
```

這段程式會先輸入學號，並檢查學號是否重複。如果重複，就不新增資料。如果沒有重複，才繼續輸入姓名與成績。

排序功能使用 `sort()` 完成：

```
sort(students.begin(), students.end(), [](const Student& a, const Student& b) {
    return a.score > b.score;
});
```

這段程式使用 lambda function 比較兩位學生的成績，讓成績高的學生排在前面。

統計功能中，我使用 `getMax` 和 `getMin` 來計算最高分與最低分：

```
highest = getMax(highest, s.score);  
lowest = getMin(lowest, s.score);
```

這部分符合本次作業要求的 template function 固定挑戰題，因為程式不只定義 template function，也有實際在小專題中呼叫使用。

六、執行畫面或輸出結果

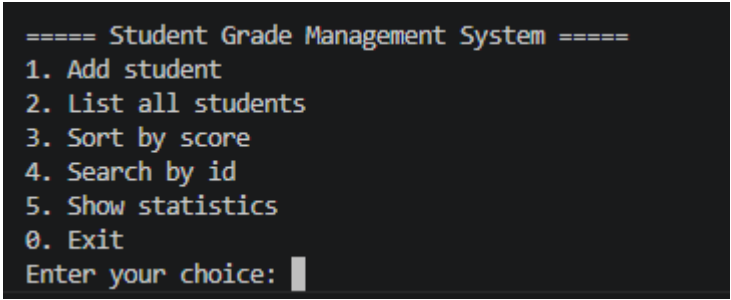
本程式成功使用以下指令編譯：

```
g++ -std=c++17 main.cpp -o main.exe
```

並使用以下指令執行：

```
.\main.exe
```

程式執行後會出現選單：

A screenshot of a terminal window showing the menu of the 'Student Grade Management System'. The menu is displayed in a monospaced font with a light blue background. It lists six options: 1. Add student, 2. List all students, 3. Sort by score, 4. Search by id, 5. Show statistics, and 0. Exit. Below the list, it prompts 'Enter your choice:' followed by a cursor.

```
===== Student Grade Management System =====  
1. Add student  
2. List all students  
3. Sort by score  
4. Search by id  
5. Show statistics  
0. Exit  
Enter your choice: █
```

測試資料如下：

A001 Tom 85

A002 Amy 92

A003 Ben 55

統計結果如下：

Average score: 77.33

Highest score: 92

Lowest score: 55

Pass students: 2

Fail students: 1


```
PS D:\666VSC> g++ -std=c++17 main.cpp -o main.exe
PS D:\666VSC> .\main.exe
```

===== Student Grade Management System =====

1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit

Enter your choice: 1

Enter student id: A001

Enter student name: Tom

Enter score: 85

Student added successfully.

===== Student Grade Management System =====

1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit

Enter your choice: 1

Enter student id: A002

Enter student name: Amy

Enter score: 92

Student added successfully.

===== Student Grade Management System =====

1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit

Enter your choice: 1

Enter student id: A003

Enter student name: Ben

Enter score: 55

Student added successfully.

===== Student Grade Management System =====

1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit

Enter your choice: 2

ID	Name	Score
----	------	-------

A001	Tom	85
A002	Amy	92
A003	Ben	55

===== Student Grade Management System =====

1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit

Enter your choice: 3

Students sorted by score from high to low.

===== Student Grade Management System =====

1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit

Enter your choice: 2

ID	Name	Score
----	------	-------

A002	Amy	92
A001	Tom	85
A003	Ben	55

===== Student Grade Management System =====

1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit

Enter your choice: 4

Enter student id to search: A002

Student found:

ID: A002

Name: Amy

Score: 92

===== Student Grade Management System =====

1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit

Enter your choice: 5

Average score: 77.33

Highest score: 92

Lowest score: 55

Pass students: 2

Fail students: 1

===== Student Grade Management System =====

1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit

Enter your choice: 0

Program ended.

PS D:\666VSC> █

七、GitHub repo link 與檔案結構說明

本次作業的 GitHub Repository 連結如下：

<https://github.com/sodaoao/student-grade-management-system>

本專案包含以下檔案：

main.cpp
README.md
report.pdf

main.cpp 是主要程式碼，包含學生成績管理系統的所有功能。

README.md 是專案說明文件，內容包含專案主題、功能介紹、使用的 STL、template function、編譯方式與執行方式。

report.pdf 是本次自主學習報告，記錄學習主題、程式設計方式、執行結果與心得。

GitHub repository 設定為 Public，方便助教可以直接查看程式碼與報告內容。

八、遇到的問題與解決方式

在完成本次小專題時，我遇到的第一個問題是剛開始不太清楚 STL 和 template 的關係。後來透過整理後，我理解到 STL 其實就是 template 的實際應用，例如 `vector<Student>` 就代表用 `vector` 這個模板容器儲存 `Student` 型別的資料。

第二個問題是在編譯時出現錯誤。剛開始程式無法成功執行，後來發現是 `main.cpp` 檔案內容沒有正確存入，導致編譯器找不到 `main()` 函式。解決方式是重新確認 `main.cpp` 內容，並確實按下存檔，再重新使用 g++ 編譯。

第三個問題是 GitHub 的使用。因為之前不熟悉 GitHub repository 的操作，所以一開始不知道如何上傳檔案。後來使用 GitHub 網頁版的 Upload files 功能，成功上傳 `main.cpp` 和 `README.md`。

透過這些問題，我學到寫程式不只是完成程式碼，也要注意檔案是否正確儲存、編譯指令是否正確，以及專案檔案是否整理完整。

九、學習心得與未來可延伸方向

透過這次自主學習，我對 C++ 的 STL 和 template 有了更清楚的理解。以前只知道陣列可以存很多資料，但這次使用 `vector` 後，發現 `vector` 可以動態增加資料，比傳統陣列更方便，也更適合實際專案。

這次也讓我練習到 `sort()` 的使用方式。以前排序可能會想到自己寫迴圈比較，但使用 STL 的 `sort()` 可以讓程式更簡潔，也更容易閱讀。

另外，template function 的部分讓我理解到，同一個函式可以支援不同型別，這是 C++ 很有彈性的地方。本次雖然只用在整數成績的最高分與最低分統計，但未來如果要比較其他型別，也可以使用相同的概念。

如果未來要延伸這個系統，可以加入更多功能，例如修改學生資料、刪除學生資料、將資料儲存到檔案、從檔案讀取學生資料，或是改用 `map` 以學號作為 key 來提升查詢效率。

整體來說，這次作業讓我從單純理解語法，進一步練習如何把 `struct`、`vector`、`sort()`、function 和 template function 整合成一個完整的小型系統，對之後學習資料結構也有幫助。